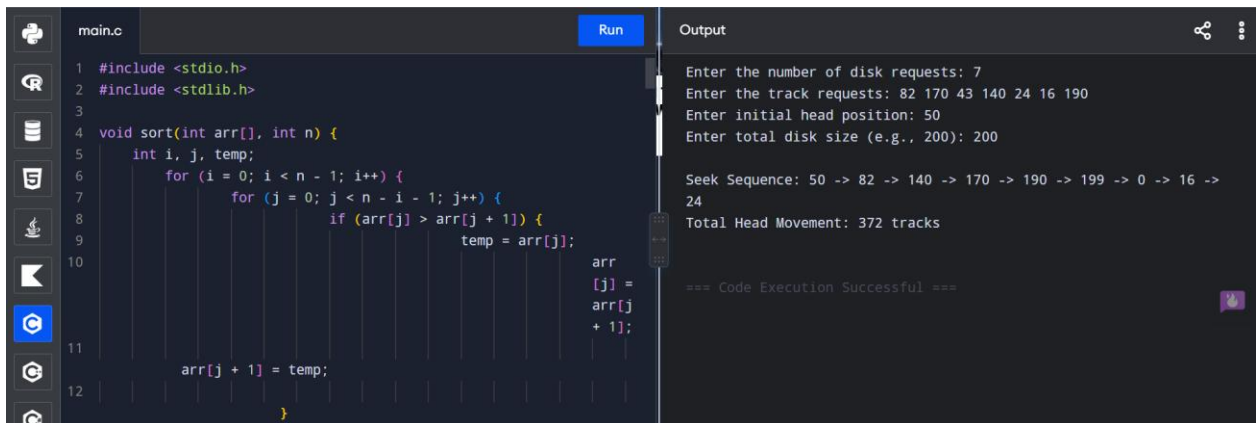


EXPERIMENT 33:

33. Develop a C program to simulate the C-SCAN (Circular SCAN) disk scheduling algorithm.



```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void sort(int arr[], int n) {
5     int i, j, temp;
6     for (i = 0; i < n - 1; i++) {
7         for (j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12            }
13        }
14    }
15}
16
17 int main() {
18     int n, i;
19     printf("Enter the number of disk requests: ");
20     scanf("%d", &n);
21     int arr[n];
22     printf("Enter the track requests: ");
23     for (i = 0; i < n; i++) {
24         scanf("%d", &arr[i]);
25     }
26     printf("Enter initial head position: ");
27     int head;
28     scanf("%d", &head);
29     printf("Enter total disk size (e.g., 200): ");
30     int disk_size;
31     scanf("%d", &disk_size);
32
33     sort(arr, n);
34
35     printf("Seek Sequence: ");
36     int current_track = head;
37     int total_head_movement = 0;
38     int i = 0;
39     while (i < n) {
40         printf("%d -> ", arr[i]);
41         total_head_movement += abs(arr[i] - current_track);
42         current_track = arr[i];
43         i++;
44     }
45     printf("%d\n", current_track);
46     printf("Total Head Movement: %d tracks\n", total_head_movement);
47
48     return 0;
49 }
```

Output

```
Enter the number of disk requests: 7
Enter the track requests: 82 170 43 140 24 16 190
Enter initial head position: 50
Enter total disk size (e.g., 200): 200

Seek Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 0 -> 16 ->
24
Total Head Movement: 372 tracks

=== Code Execution Successful ===
```

PSEUDO CODE :

- Read the number of disk requests (N), request track array, initial head position ($head$), total disk size ($disk_size$), and head movement direction (towards higher or lower track values).
- Add boundary tracks (0 and $disk_size - 1$) to the request list.
- Sort all disk tracks in ascending order.
- Locate the index of the initial head position in the sorted tracks.
- If the direction is towards higher tracks:
 - Move from the current head index up to the maximum track ($disk_size - 1$), adding $|current_track - previous_track|$ to total head movement.
 - Jump directly from maximum track ($disk_size - 1$) to track 0 without counting head movement for the wrap-around (or counting total disk size depending on standard convention; standard C-SCAN jumps to track 0).
 - Continue servicing from track 0 up to the track just before the original head position.
- Print the sequence of track servicing and the calculated total head movement.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void sort(int arr[], int n)
```

```
{
```

```
    int i, j, temp;
```

```
    for (i = 0; i < n - 1;  
i++) {
```

```
        for (j = 0; j < n - i -  
1; j++) {
```

```
            if (arr[j] > arr[j +  
1]) {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j +  
1];
```

```
                arr[j + 1] =  
temp;
```

```
    }
```

```
    }  
}  
}
```

```
int main() {  
  
    int queue[100],  
    req[100];  
  
    int n, head, disk_size,  
    total_movement = 0;  
  
    int index, i, current;  
  
    printf("Enter the  
number of disk requests:  
");  
  
    scanf("%d", &n);  
  
    printf("Enter the track  
requests: ");  
  
    for (i = 0; i < n; i++)
```

```
{

    scanf("%d",
&req[i]);

}


printf("Enter initial
head position: ");

scanf("%d", &head);


printf("Enter total
disk size (e.g., 200): ");

scanf("%d",
&disk_size);


// Copy requests and
include disk boundaries

for (i = 0; i < n; i++)
{

    queue[i] = req[i];
```

```

    }

    queue[n] = 0;

    queue[n + 1] =
disk_size - 1;

    n += 2;


    // Sort the requests

    sort(queue, n);


    // Find head position
in sorted array

    for (i = 0; i < n; i++)
    {

        if (head < queue[i])
        {

            index = i;

            break;

        }

    }
}

```

```
printf("\nSeek  
Sequence: %d", head);  
  
current = head;  
  
// Moving towards  
right (higher numbers)  
  
for (i = index; i < n;  
i++) {  
  
    printf(" -> %d",  
queue[i]);  
  
    total_movement +=  
abs(queue[i] - current);  
  
    current = queue[i];  
  
}  
  
// Jump to the  
beginning track 0  
  
total_movement +=
```

```
abs((disk_size - 1) - 0);
```

```
current = 0;
```

```
printf(" -> %d",  
current);
```

```
// Continue servicing  
tracks from left up to the  
index before starting  
head
```

```
for (i = 0; i < index -  
1; i++) {
```

```
// Skip track 0  
since we jumped there
```

```
if (queue[i] == 0)  
continue;
```

```
printf(" -> %d",  
queue[i]);
```

```
total_movement +=  
abs(queue[i] - current);
```

```
current = queue[i];
```

```
}

printf("\nTotal Head
Movement: %d
tracks\n",
total_movement);

return 0;

}
```

INPUT :

Enter the number of disk
requests: 7

Enter the track requests:
82 170 43 140 24 16
190

Enter initial head
position: 50

Enter total disk size
(e.g., 200): 200

OUTPUT:

Seek Sequence: 50 ->

82 -> 140 -> 170 -> 190

-> 199 -> 0 -> 16 -> 24

-> 43

Total Head Movement:

382 tracks